

SauceDemo Test Framework – Playwright + Cucumber + TypeScript

A Page-Object-Model UI test suite converted from a Selenium + Cucumber-JVM (Java) framework. Same Gherkin scenarios, same site under test (saucedemo.com) — the automation layer underneath is now Playwright, orchestrated by Cucumber-js in TypeScript.

17

SCENARIOS

91

STEPS

10

PAGE OBJECTS

5

FEATURE FILES

4

BROWSER OPTIONS

01 – STACK

What replaced what

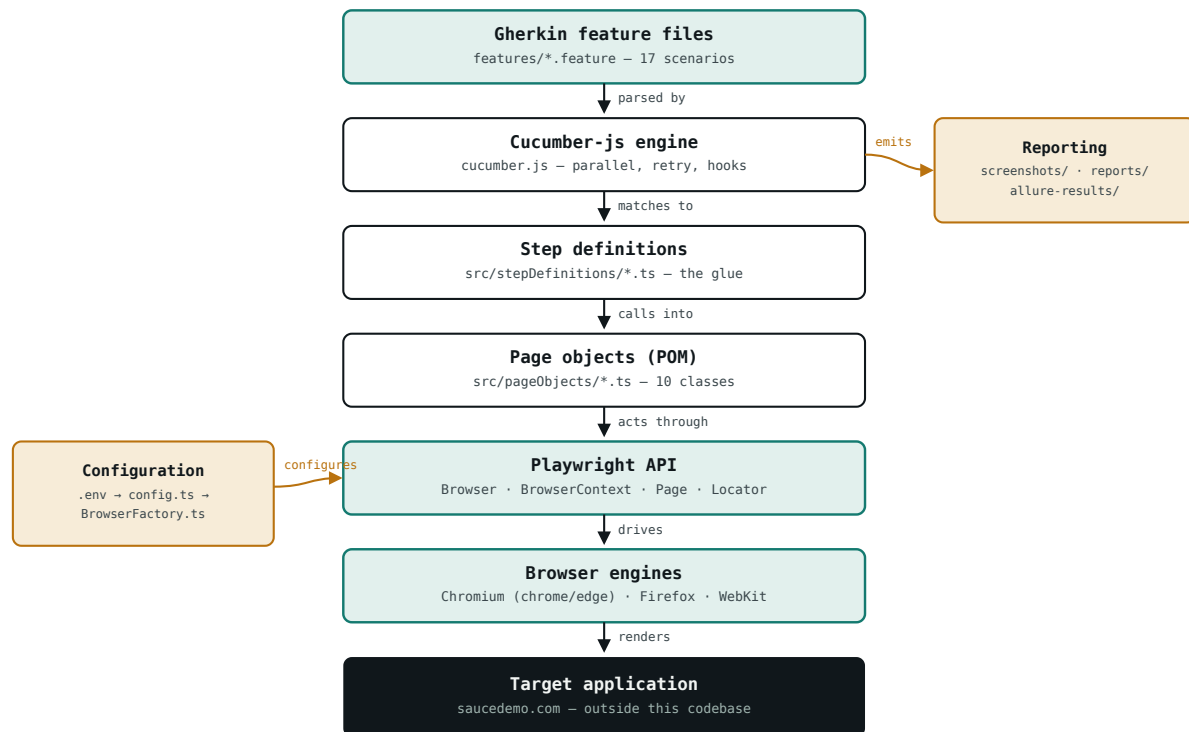
The Gherkin layer didn't change at all — `.feature` files are engine-agnostic. Everything below the Gherkin layer was rebuilt.

CONCERN	ORIGINAL (JAVA)	THIS FRAMEWORK (TYPESCRIPT)
Browser driver	Selenium WebDriver	Playwright
BDD runner	Cucumber-JVM + JUnit	@cucumber/cucumber (cucumber-js)
Build tool	Maven (pom.xml)	npm (package.json)
Driver factory	factory/BaseClass.java	src/factory/BrowserFactory.ts
Config source	config.properties	.env
Page objects	pageObject/*.java (@FindBy)	src/pageObjects/*.ts (Locator)
Step definitions	stepDefination/*.java	src/stepDefinitions/*.ts
Setup/teardown	Hooks.java	src/stepDefinitions/hooks.ts
Runner config	TestRunner.java (@CucumberOptions)	cucumber.js
Explicit waits	WebDriverWait + JS-click retries	built-in Playwright auto-wait

02 – SYSTEM ARCHITECTURE

Block diagram

Six layers, stacked in the order a scenario actually passes through them, with configuration feeding in on one side and reporting draining out the other.



Teal layers are what SauceDemo's original Selenium stack also had in some form. Step definitions and page objects (white) are the layers that were rebuilt in TypeScript.

Figure 1. The framework as six stacked layers. Configuration flows in from the side to shape how the Playwright layer launches a browser; every result flows out the other side into the report and screenshot formats covered in §09.

03 - LAYOUT

Folder structure

```

playwright-cucumber-saucedemo/
├─ .env                # EXECUTION_ENV, BROWSER, HEADLESS, APP_URL, SLOW_MO, HOLD_MS
├─ cucumber.js         # glue, formatters, parallel:2, retry:1, timeout:20000
├─ package.json
├─ tsconfig.json
├─ features/
│  ├─ login.feature    # 12 scenarios
│  ├─ sort.feature     # 1 scenario · 4 sort checks
│  ├─ hamberger.feature # 2 scenarios
│  ├─ footer.feature   # 1 scenario
│  └─ endtoendflow.feature # 1 outline · 1 example row
├─ src/
│  ├─ config/config.ts
│  ├─ factory/BrowserFactory.ts
│  ├─ world/CustomWorld.ts
│  ├─ pageObjects/     # 10 classes - see §07
│  └─ stepDefinitions/ # hooks.ts + 5 step files - see §07
├─ reports/            # generated: cucumber-report.html / .json
├─ screenshots/        # generated: one PNG per Then-step + failures
└─ allure-results/     # generated: raw Allure result files
  
```

04 – DATA FLOW

How a scenario reaches the real browser

Six hops from a line of Gherkin to a real click in a real, running browser against saucedemo.com.

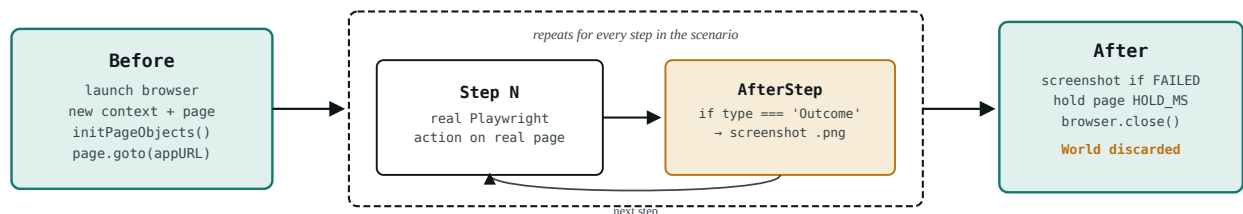


Figure 2. A Gherkin line is parsed into a step, matched against the registered patterns in every `stepDefinitions/*.ts` file, and the matching function — bound to that scenario's `CustomWorld` — calls a method on a page object, which acts on a real Playwright page against the real, running site.

05 – LIFECYCLE

What happens once, per scenario

Every one of the 17 scenarios runs this exact sequence independently — a fresh browser is born and destroyed each time, so nothing ever leaks between tests.



Config behind this run: `timeout: 20000 (cucumber.js) · parallel: 2 · retry: 1`

Figure 3. `hooks.ts` 's three lifecycle functions. Because `parallel: 2` is set in `cucumber.js`, two of these full cycles run side by side in separate worker processes rather than strictly one after another.

CustomWorld and the page objects

Cucumber-js builds one fresh `CustomWorld` instance per scenario. It's the only object every hook and step function receives as `this` — and it's what wires the live browser page to every page object.

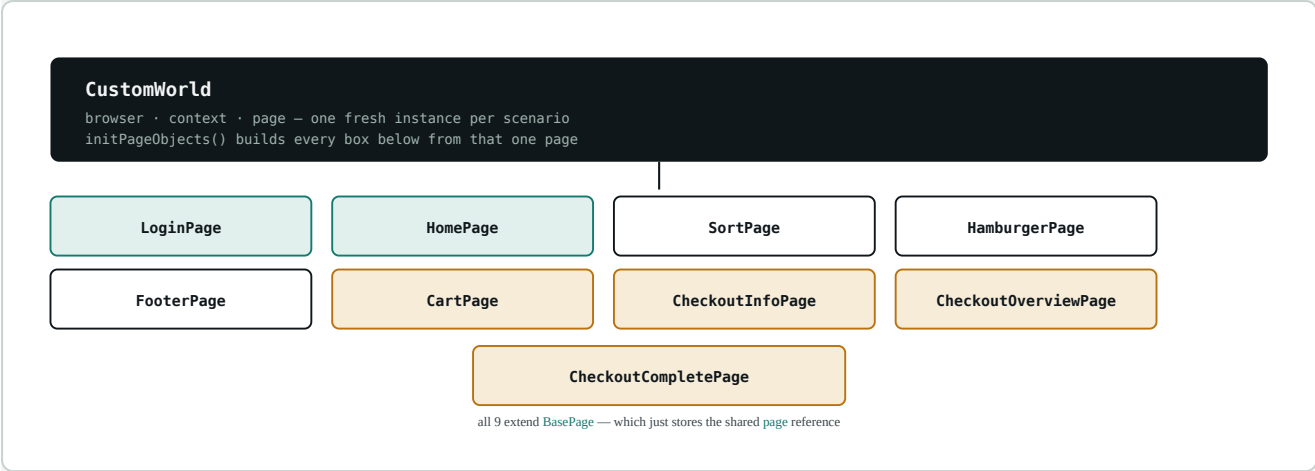


Figure 4. Amber boxes (`CartPage` , the three checkout pages) run only inside `endtoendflow.feature`’s single scenario outline. Teal boxes (`LoginPage` , `HomePage`) are exercised by nearly every feature, since every scenario’s `Background` logs in first.

Page objects

CLASS	RESPONSIBILITY	EXERCISED BY
BasePage	Shared parent — holds the <code>page</code> reference every subclass needs.	all
LoginPage	Username/password fields, login button, error message, logo.	login.feature + every Background
HomePage	Products page: title, cart icon, cart badge, add-to-cart.	login · sort · hamberger · endtoendflow
SortPage	Sort dropdown; verifies name/price ordering client-side.	sort.feature
HamburgerPage	Side menu: All Items, About, Logout, Reset App State.	hamberger.feature
FooterPage	Copyright text, policy links, social icons.	footer.feature
CartPage	Cart contents, checkout button.	endtoendflow.feature
CheckoutInfoPage	First/last name + zip form, continue button.	endtoendflow.feature
CheckoutOverviewPage	Order review, finish button.	endtoendflow.feature
CheckoutCompletePage	Confirmation header, back-to-products button.	endtoendflow.feature

Step definitions

FILE	COVERS
hooks.ts	Before / After / AfterStep — browser lifecycle + screenshots (see Figure 3).
loginSteps.ts	Every step in <code>login.feature</code> 's 12 scenarios.
sortSteps.ts	Opening the sort filter, selecting an option, verifying order.
hamburgerSteps.ts	Menu open/click, About round-trip, logout redirect, Reset-App-State cart-badge checks.
footerSteps.ts	Scroll-to-footer, copyright/policy text, social links.
endToEndFlowSteps.ts	Add-to-cart → checkout → confirmation → back-to-home, full path.

08 - CONFIGURATION

.env → config.ts → BrowserFactory.ts

One `.env` file is read once at process start by `dotenv.config()`, validated and defaulted by `config.ts`, then consumed by `BrowserFactory.ts` at every browser launch.

.ENV KEY	DEFAULT	USED BY	BROWSER VALUE	PLAYWRIGHT CALL
EXECUTION_ENV	local	guards the unwired "remote" path	chrome	chromium.launch()
BROWSER	chrome	chrome · edge · firefox · webkit	edge	chromium.launch({channel: 'msedge'})
			firefox	firefox.launch()
HEADLESS	true	every <code>.launch()</code> call	webkit	webkit.launch()
APP_URL	saucedemo.com	<code>Before</code> hook's <code>page.goto()</code>	Edge isn't a separate engine. Edge and Chrome both run on Chromium, so Playwright launches the real installed Edge binary through the <code>chromium</code> launcher's <code>channel</code> option rather than a dedicated "Edge" type.	
SLOW_MO	400	ms delay Playwright adds per action		
HOLD_MS	5000	<code>After</code> hook — keeps page open post-scenario		

npm scripts

COMMAND	RUNS
<code>npm test</code>	all 5 feature files — 17 scenarios
<code>npm run test:login</code>	login.feature only — 12 scenarios
<code>npm run test:sort</code>	sort.feature only
<code>npm run test:hamburger</code>	hamburger.feature only
<code>npm run test:footer</code>	footer.feature only
<code>npm run test:e2e</code>	endtoendflow.feature only
<code>npm run allure:report</code>	builds + opens the Allure HTML dashboard

Every run, driven by `cucumber.js`

- 1 **parallel: 2** — two scenarios run at once, in separate worker processes.
- 2 **retry: 1** — a scenario that fails is re-run once automatically before being reported as a real failure.
- 3 **timeout: 20000** — every step and every hook gets 20s before Cucumber itself gives up, independent of Playwright's own 20s default action timeout set in `hooks.ts`.
- 4 **format** — progress-bar to the terminal, plus HTML, JSON, and Allure reports written to `reports/` and `allure-results/`.